



Dall'AI che scrive codice all'AI che lo esegue

senza far esplodere il backend





Oggi un LLM genera codice senza sforzo

«Analizza questo file PDF e restituiscimi un Excel»



Python



SQL



Bash



Programmi completi

La domanda cambia

«Come genero il codice?»



«Dove lo eseguo?»

Generare codice è risolto. Eseguirlo in sicurezza, no.



Strumenti che usiamo già ogni giorno eseguono codice



ChatGPT

Advanced Data Analysis



Claude Code

loop di esecuzione nel
terminale



GitHub Copilot Agent

esegue, testa, corregge



Azure AI Foundry

Code Interpreter



Agenti di data analysis

script generati al volo

Se l'AI deve trasformare dati, verificare risultati o lavorare sui file degli utenti, prima o poi deve eseguire del codice.

SaaS Runtime vs Application Runtime



SaaS Runtime

l'esecuzione avviene dentro la piattaforma

ChatGPT

Foundry Code Interpreter

Claude



Application Runtime

l'esecuzione avviene nella tua applicazione

Semantic Kernel

Agent custom

Workflow AI aziendali

Pattern 1 — Generated Script



CHI ORCHESTRA

Nessuno: l'utente decide ed esegue

QUANTE ESECUZIONI

Una sola, sincrona

Pattern 2 — Generated Worker



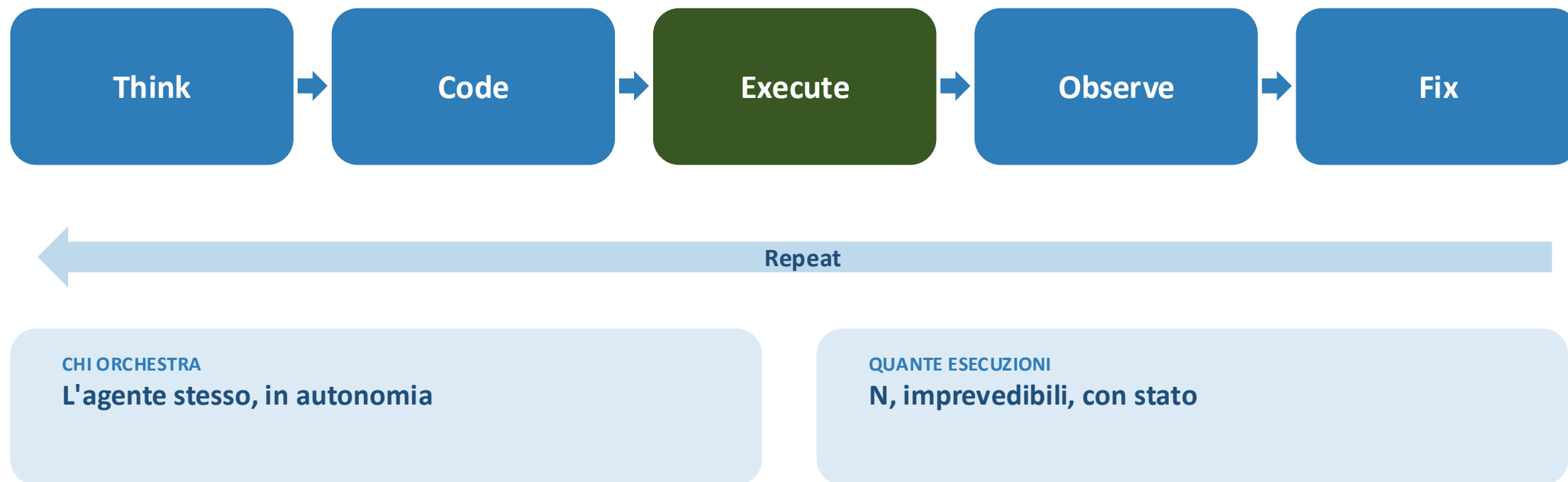
CHI ORCHESTRA

L'applicazione (Semantic Kernel, LangChain...)

QUANTE ESECUZIONI

Poche, pilotate dal codice

Pattern 3 — Agentic Execution Loop



Lo stato deve sopravvivere tra un'esecuzione e l'altra: qui il runtime diventa critico.

Where does the code run?



Runtime per eseguire il codice generato



sicurezza



isolamento



dipendenze



multi-tenancy



cleanup



scalabilità

Possibili soluzioni



Process.Start()

La soluzione «YOLO»: nessun isolamento, nessun controllo



Containers

Funziona, ma devi gestire immagini, lifecycle, cleanup, osservabilità, pooling



VM dedicate

Funzionano, ma costi e tempi crescono rapidamente



Azure Container Apps Dynamic Sessions

Non voglio creare container. Voglio eseguire codice in modo sicuro.

Runtime as a Service



Isolamento a livello di
hypervisor



Scalabilità e session
pool

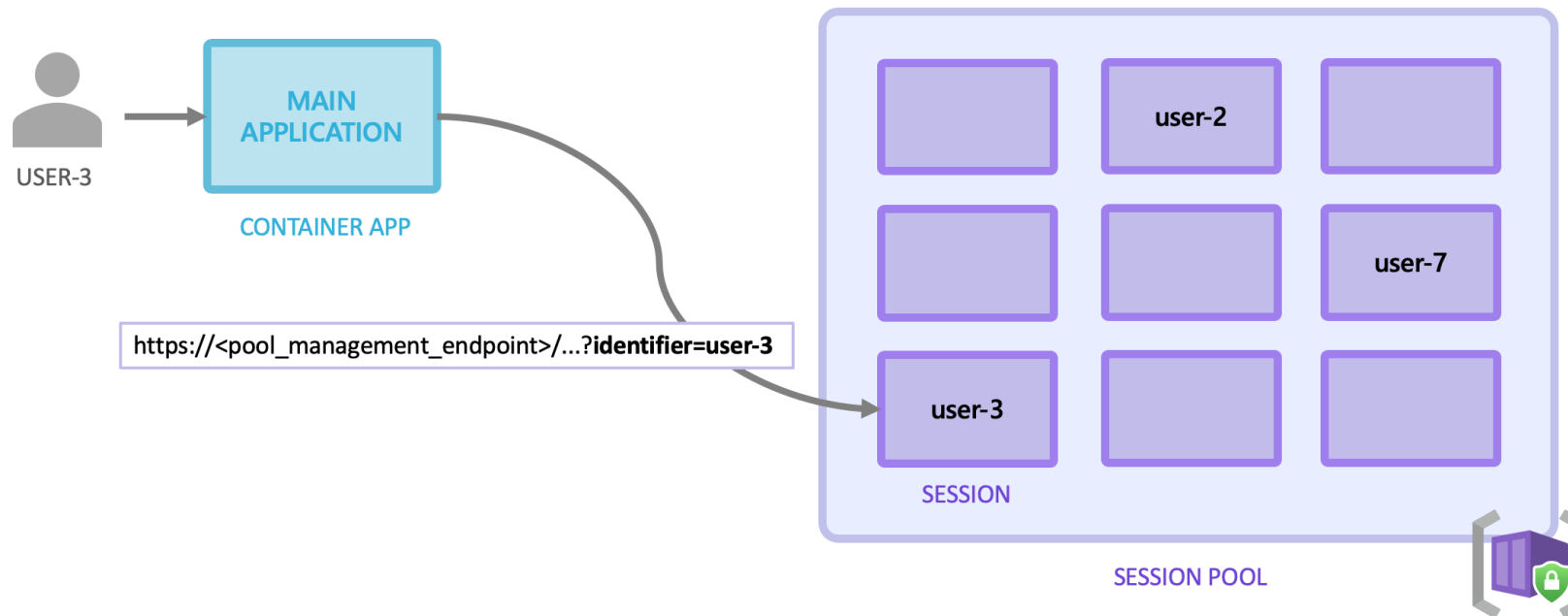


Lifecycle e cleanup
automatici



Nessuna immagine da
mantenere

Come funziona: session pool e sessioni



Il session identifier decide in quale sessione finisce il codice: stesso id → stessa sessione e stesso stato.
Ogni sessione è isolata da un confine Hyper-V, pensata per eseguire codice non fidato.
Il pool scala a centinaia/migliaia di sessioni concorrenti senza interventi manuali.
Le sessioni sono pre-riscaldate: allocazione in millisecondi, deprovisioning automatico dopo il cooldown.

Integrazione e scenari tipici



REST API

esegui codice, carica e scarica file



Framework LLM

LangChain, LlamaIndex, Semantic Kernel



Codice generato da LLM

eseguito senza rischi per i tuoi sistemi

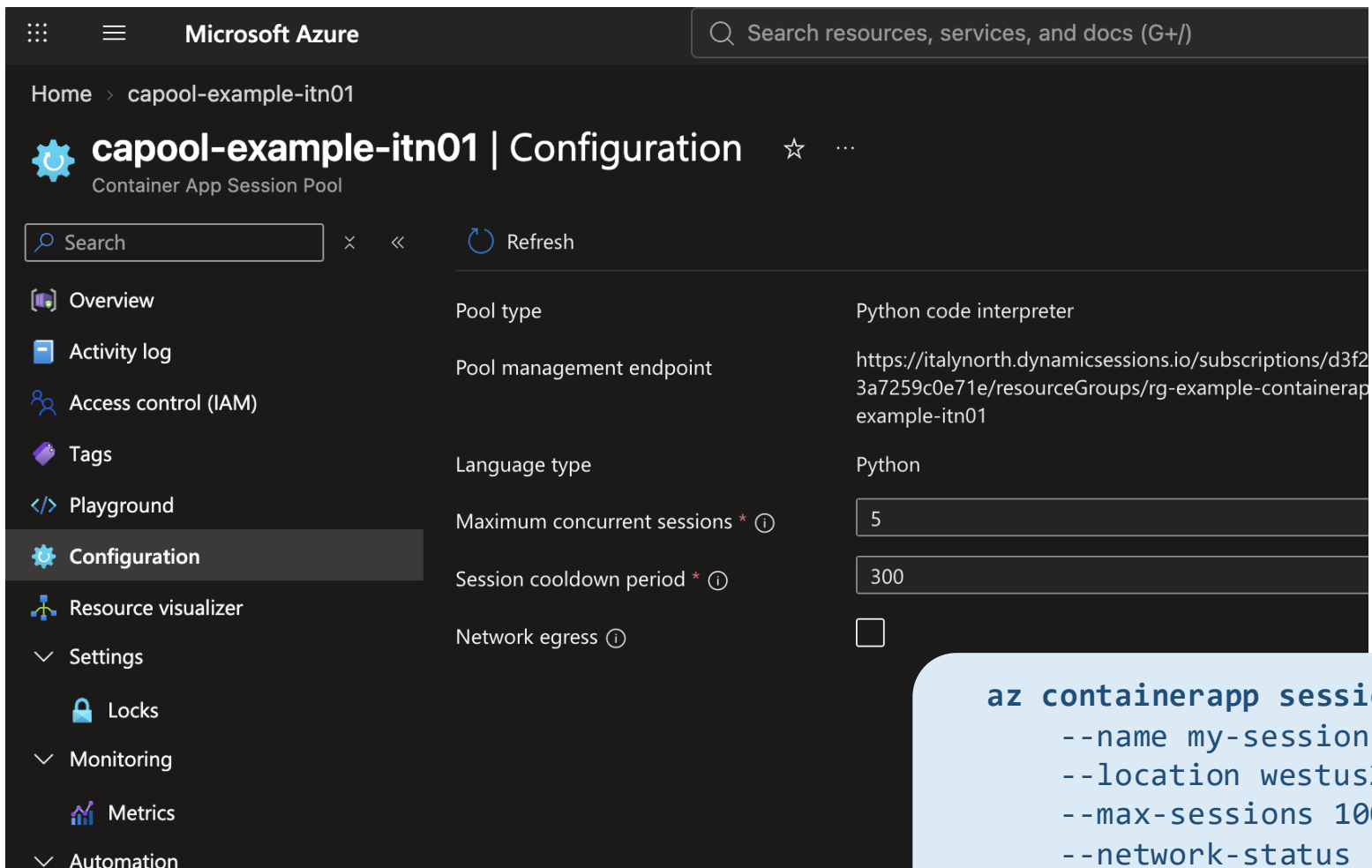


Codice utente non fidato

sandbox per app web e multi-tenant

Il pattern è sempre lo stesso: l'app genera il codice, la sessione lo esegue in sandbox.

Creare un session pool: Portale o Azure CLI



The screenshot shows the Azure portal interface for a Container App Session Pool. The left sidebar contains navigation links: Overview, Activity log, Access control (IAM), Tags, Playground, Configuration (selected), Resource visualizer, Settings, Locks, Monitoring, Metrics, and Automation. The main content area displays the configuration for 'capool-example-itn01'. The configuration table is as follows:

Pool type	Python code interpreter
Pool management endpoint	https://italynorth.dynamicsessions.io/subscriptions/d3f23a7259c0e71e/resourceGroups/rg-example-containerapp-example-itn01
Language type	Python
Maximum concurrent sessions *	5
Session cooldown period *	300
Network egress	☐

Anche dal portale dedicato

containerapps.azure.com

La console dedicata ad Azure Container Apps: crei e gestisci il session pool e provi le sessioni dal browser.



New

```
az containerapp sessionpool create \  
  --name my-session-pool --resource-group <RESOURCE_GROUP> \  
  --location westus2 --container-type PythonLTS \  
  --max-sessions 100 --cooldown-period 300 \  
  --network-status EgressDisabled
```

Come si usa: management endpoint e identifier

```
POST <POOL_MANAGEMENT_ENDPOINT>/<API_PATH>?identifier=<USER_ID>
Authorization: Bearer <TOKEN>
{ "command": "echo 'Hello, world!'" }
```

1 • Crea il session pool

Definisci immagine, numero massimo di sessioni e idle timeout.

2 • Autenticati

Token Microsoft Entra + ruolo Azure ContainerApps Session Executor sul pool.

3 • Chiama con un identifier

Tutto ciò che segue l'endpoint viene inoltrato al container della sessione.

Nessuna container app da deployare: crei il session pool e chiami la sua management API.

Eseguire codice: la chiamata /executions

```
POST https://<REGION>.dynamicsessions.io/subscriptions/<SUB_ID>/resourceGroups/<RG>/  
      sessionPools/<POOL>/executions?api-version=2025-10-02-  
      preview&identifier=<SESSION_ID>  
Content-Type: application/json  
Authorization: Bearer <TOKEN>
```

```
{ "properties": {  
  "codeInputType": "inline",  
  "executionType": "synchronous",  
  "code": "print('Hello, world!')" } }
```

codeInputType: inline

il codice viaggia nel body della richiesta

executionType: synchronous

rispondi subito con stdout e risultato

identifier

sceglie la sessione in cui eseguire

Una sola POST: il codice entra, il risultato torna indietro.

File in ingresso e in uscita dalla sessione

POST	<ENDPOINT>/files/upload?identifier=<USER_ID>	(multipart/form-data)
GET	<ENDPOINT>/files?identifier=<USER_ID>	elenca i file della sessione
GET	<ENDPOINT>/files/content/report.xlsx?identifier=<USER_ID>	scarica il file

1 • Carica

Il PDF o il CSV dell'utente entra nella sessione, in /mnt/data.

2 • Elabora

Il codice generato legge il file e produce un nuovo artefatto.

3 • Scarica

L'app recupera l'Excel prodotto e lo restituisce all'utente.

I file vivono nella sessione: stesso identifier, stessi file; sessione chiusa, tutto sparisce.

Gestione del ritorno: successo ed errori

200 OK — esecuzione riuscita

```
{
  "properties": {
    "status": "Success",
    "stdout": "hello-world\n",
    "stderr": "",
    "executionResult": "",
    "executionTimeInMilliseconds": 5
  }
}
```

4xx / 5xx — risposta di errore

```
{
  "error": {
    "code": "ErrorCode",
    "message": "Human-readable description",
    "details": "Optional additional context",
    "target": "Field or parameter in error",
    "traceId": "Request trace ID for debugging"
  }
}
```

SessionWithIdentifierNotFound
400 — sessione inesistente o scaduta:
verifica l'identifier

SessionRequestValidationFailed
400 — campi o query param non validi
(identifier, skip, api-version)

SessionRequestNotSupported
400 — endpoint o metodo non
supportato dall'API

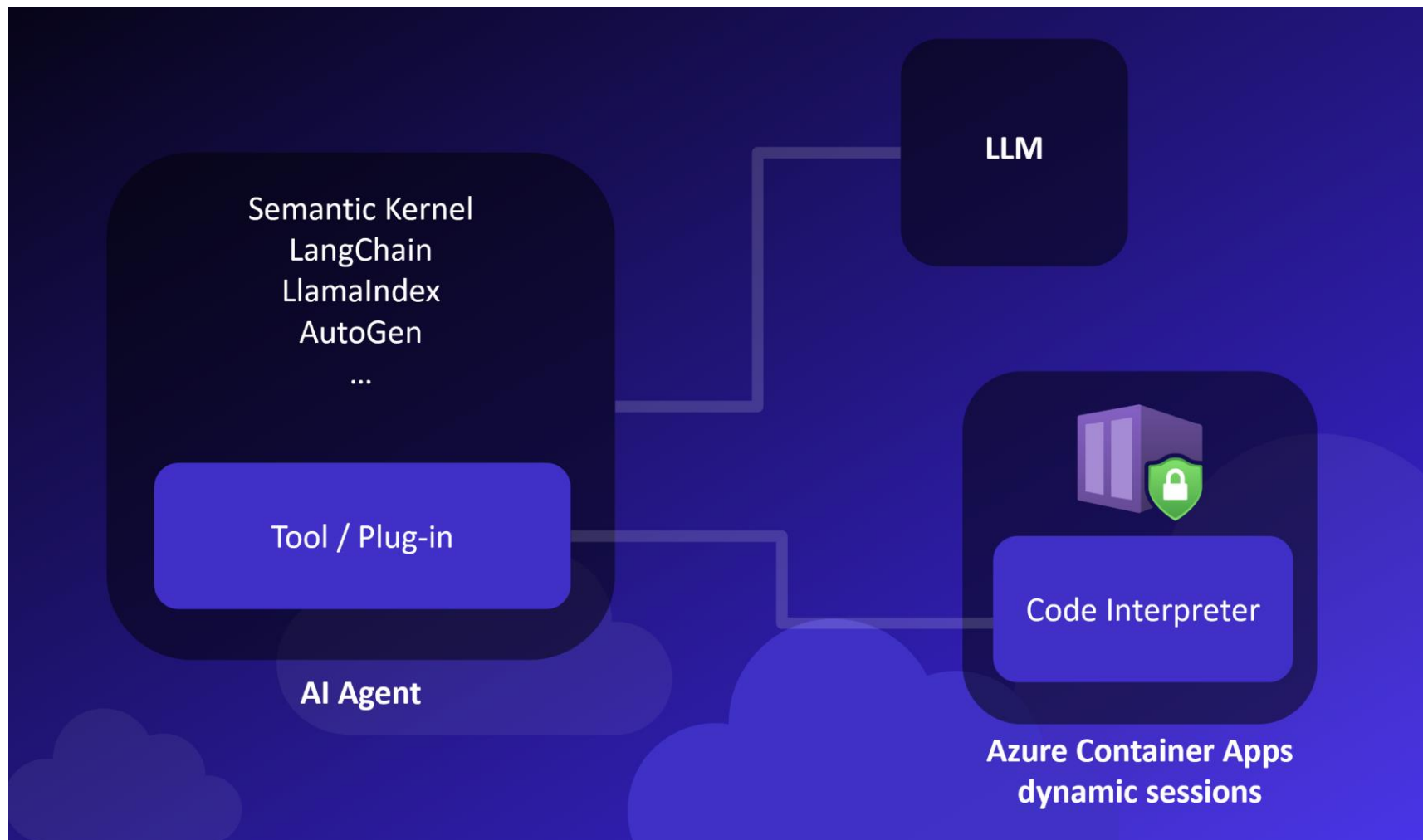
InternalServerError
500 — errore lato server: ritenta e
controlla il traceId



Demo



Integrazioni



Semantic Kernel: il plugin è già pronto

```
// 1. Configura il plugin: punta al pool ACA Dynamic Sessions e alla sessione isolata
var sessionId = Guid.NewGuid().ToString("N");
var settings = new SessionsPythonSettings(
    sessionId,
    endpoint: new Uri("https://<region>.dynamicsessions.io/subscriptions/.../pools/<pool>"))
{
    TimeoutInSeconds = 60
};
var sessionsPlugin = new SessionsPythonPlugin(
    settings,
    httpClientFactory,
    authTokenProvider: ct => GetAzureTokenAsync(credential, ct), // token AAD
    loggerFactory);
// 2. Registra il plugin nel Kernel: diventa la funzione "execute_python"
var kernel = Kernel.CreateBuilder()
    .AddAzureOpenAIChatCompletion(deployment, endpoint, credential)
    .Build();
kernel.Plugins.AddFromObject(sessionsPlugin, "SessionsPython");
```

SessionsPythonPlugin

il tool ufficiale per ACA Dynamic Sessions

AddFromObject

registrato nel Kernel come
execute_python

sessionId

un identifier per utente: stato isolato e
persistente



Demo



Due modalità di sessione

Code interpreter sessions

- Interprete pronto all'uso
- Upload e download di file nella sessione
- REST API + tool per LangChain, LlamaIndex, Semantic Kernel
- Nessuna immagine o infrastruttura da gestire

Tre interpreti built-in, pronti all'uso

Py Python

JS Node.js

>_ Shell

Custom container sessions

- La tua immagine container nel pool
- Controllo totale su runtime, librerie, tool
- Per linguaggi o dipendenze non coperti dal built-in
- Stessa isolation e stesso ciclo di vita gestito

Parametri del session pool



Maximum concurrent sessions

5

Quante sessioni possono essere attive contemporaneamente nel pool: è il limite di parallelismo (e di costo).



Session cooldown period

300 s

Per quanto la sessione resta viva dopo l'ultima richiesta, prima di essere distrutta. Stato conservato, riuso immediato.



Network egress

disabilitato

Il codice generato può uscire in rete? Di default no: massimo isolamento per codice non fidato.

Tre parametri e il runtime è configurato: quanta concorrenza, quanto dura la sessione, quanto isolamento.

Quanto costa



Platform-managed container

\$0,03 / ora di sessione

- Immagini pronte: Python, Node.js, Shell
- Si paga solo il tempo in cui la sessione è attiva o in cooldown
- Sconti progressivi sui volumi elevati
- Nessuna infrastruttura da dimensionare



Custom container

Dedicated plan (CPU + memoria)

- Fatturato sulle risorse di calcolo del pool e delle sessioni attive
- Istanze dedicate (tipicamente E16)
- Allocazione in base alle sessioni pronte o in esecuzione
- Più controllo, costo a capacità

<https://sandboxes.azure.com>



Sandboxes

Isolated, on-demand microVMs - built for AI agents, code execution, and dev environments.



Get Started



View Docs



Secure egress. Built-in egress proxy with configurable deny policies, host allowlists, and credential injection.



Strong isolation. Each sandbox runs in its own secure boundary with its own kernel. Safe for untrusted code.



Built for AI Apps & Agents. Programmable from CLI, Python SDK, or REST - the shape AI workloads need.



Sub-second startup. Boot, execute, and discard a sandbox in seconds.



Suspend and resume. Snapshot full memory and disk state. Resume in sub-second, scale to zero between.



Bring your own images. Create sandboxes from container images you already use.

Sandboxes (preview)

Dynamic Sessions

GA

- Slot di esecuzione gestito dal pool
- Sessione effimera, identificata da un session id
- Il pool gestisce scaling e cleanup
- Ideale per eseguire codice generato dall'LLM

Sandboxes

PREVIEW

- Istanze di compute controllate singolarmente
- Lifecycle esplicito: suspend, snapshot, volumi, file, porte
- Ambienti programmabili per agenti long-running
- Quando lo stato di ogni ambiente conta per la tua app

Stesso obiettivo — isolare l'esecuzione — con due livelli di controllo sul ciclo di vita.

Risorse utili



Documentazione ufficiale – Serverless code interpreter sessions

learn.microsoft.com/azure/container-apps/sessions



Panoramica dei session pool in Azure Container Apps

learn.microsoft.com/azure/container-apps/sessions-code-interpreter



Custom container sessions: creare e gestire un pool

learn.microsoft.com/azure/container-apps/sessions-custom-container



Sorgente applicativo demo usato nella sessione

github.com/bortolin/ACASessionPoolExamples



Custom code interpreter per gli agenti di Azure AI Foundry

learn.microsoft.com/azure/foundry/agents/how-to/tools/custom-code-interpreter



Marco Bortolin

email: m.bortolin@hunext.com

twitter: @marcobortolin

<https://github.com/bortolin>

<https://www.linkedin.com/in/marcobortolin>

